



# Calories Prediction of Indian Food Using Decision Tree and Random Forest Regression Models

## Problem Statement:

With increasing awareness about nutrition and healthy eating, estimating the calorie content of food items has become essential. This project aims to predict the calorie content of Indian food using Decision Tree and Random Forest regression models by analyzing nutritional attributes such as carbohydrates, protein, fats, sugar, fibre, and micronutrients. The study also compares the performance of single-tree and ensemble-based regression models.

## Steps to be followed

1. Importing Required Libraries
2. Load Dataset
3. Data Preprocessing
4. Checking Null values
5. Data Wrangling (Filling Missing values)
6. Data Visualization
7. Feature Selection (x,y)
8. Train-Test Split
9. Feature Scaling
10. Decision Tree Model Training and Prediction
11. Model Evaluation (R2 Score,MAE,MSE,RMSE)
12. Pruning
13. GridSearchCV
14. Scatter Plot (Actual vs Predicted values)
15. Improving Model Performance with Advanced Algorithms

## Importing Libraries

In [ ]:

```
In [47]: import numpy as np
import pandas as pd
import seaborn as sns
```

```
import matplotlib.pyplot as plt
import plotly.express as px
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_
from sklearn.preprocessing import StandardScaler, LabelEncoder, OneHotEncoder,
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score, ro
import warnings
warnings.filterwarnings('ignore')
```

In [ ]:

## Loading the Dataset

```
In [2]: df=pd.read_csv(r"C:\Users\HP\Downloads\indan food nutrition processed.csv")
df
```

Out[2]:

|      | Dish Name                                         | Calories (kcal) | Carbohydrates (g) | Protein (g) | Fats (g) | Free Sugar (g) | Fibre (g) | Sodium (mg) |  |
|------|---------------------------------------------------|-----------------|-------------------|-------------|----------|----------------|-----------|-------------|--|
| 0    | Hot tea (Garam Chai)                              | 16.14           | 2.58              | 0.39        | 0.53     | 2.58           | 0.00      | 3.12        |  |
| 1    | Instant coffee                                    | 23.16           | 3.65              | 0.64        | 0.75     | 3.62           | 0.00      | 4.92        |  |
| 2    | Espresso coffee                                   | 51.54           | 6.62              | 1.75        | 2.14     | 6.53           | 0.00      | 13.98       |  |
| 3    | Iced tea                                          | 10.34           | 2.70              | 0.03        | 0.01     | 2.70           | 0.00      | 0.23        |  |
| 4    | Raw mango drink (Aam panna)                       | 35.92           | 9.05              | 0.16        | 0.03     | 7.49           | 0.61      | 79.82       |  |
| ...  | ...                                               | ...             | ...               | ...         | ...      | ...            | ...       | ...         |  |
| 1009 | Sweet and sour tomato pickle (Khatta meetha ta... | 60.88           | 6.55              | 1.26        | 3.24     | 4.31           | 2.20      | 1281.94     |  |
| 1010 | Jhatpat achar with carrot (Jhatpat achaar gaja... | 91.21           | 6.32              | 1.98        | 6.55     | 3.04           | 5.08      | 2067.33     |  |
| 1011 | Tomato chutney (Tamatar ki chutney)               | 176.07          | 31.85             | 0.97        | 6.01     | 30.02          | 1.49      | 823.65      |  |
| 1012 | Tomato ketchup                                    | 33.07           | 6.48              | 0.91        | 0.30     | 4.68           | 1.90      | 177.97      |  |
| 1013 | Bengal 5 Spice Blend (Panch Phoran)               | 289.79          | 20.00             | 18.26       | 22.16    | 1.40           | 18.40     | 51.43       |  |

1014 rows × 12 columns

In [ ]:

# Data Preprocessing

In [3]: `df.head()`

Out[3]:

|   | Dish Name                   | Calories (kcal) | Carbohydrates (g) | Protein (g) | Fats (g) | Free Sugar (g) | Fibre (g) | Sodium (mg) | Calcium (mg) |
|---|-----------------------------|-----------------|-------------------|-------------|----------|----------------|-----------|-------------|--------------|
| 0 | Hot tea (Garam Chai)        | 16.14           | 2.58              | 0.39        | 0.53     | 2.58           | 0.00      | 3.12        | 14.0         |
| 1 | Instant coffee              | 23.16           | 3.65              | 0.64        | 0.75     | 3.62           | 0.00      | 4.92        | 20.0         |
| 2 | Espresso coffee             | 51.54           | 6.62              | 1.75        | 2.14     | 6.53           | 0.00      | 13.98       | 58.0         |
| 3 | Iced tea                    | 10.34           | 2.70              | 0.03        | 0.01     | 2.70           | 0.00      | 0.23        | 1.0          |
| 4 | Raw mango drink (Aam panna) | 35.92           | 9.05              | 0.16        | 0.03     | 7.49           | 0.61      | 79.82       | 7.0          |

In [4]: `df.tail()`

Out[4]:

|      | Dish Name                                         | Calories (kcal) | Carbohydrates (g) | Protein (g) | Fats (g) | Free Sugar (g) | Fibre (g) | Sodium (mg) | Cal |
|------|---------------------------------------------------|-----------------|-------------------|-------------|----------|----------------|-----------|-------------|-----|
| 1009 | Sweet and sour tomato pickle (Khatta meetha ta... | 60.88           | 6.55              | 1.26        | 3.24     | 4.31           | 2.20      | 1281.94     |     |
| 1010 | Jhatpat achar with carrot (Jhatpat achaar gaja... | 91.21           | 6.32              | 1.98        | 6.55     | 3.04           | 5.08      | 2067.33     |     |
| 1011 | Tomato chutney (Tamatar ki chutney)               | 176.07          | 31.85             | 0.97        | 6.01     | 30.02          | 1.49      | 823.65      |     |
| 1012 | Tomato ketchup                                    | 33.07           | 6.48              | 0.91        | 0.30     | 4.68           | 1.90      | 177.97      |     |
| 1013 | Bengal 5 Spice Blend (Panch Phoran)               | 289.79          | 20.00             | 18.26       | 22.16    | 1.40           | 18.40     | 51.43       |     |

In [5]: `df.sample()`

Out[5]:

|     | Dish Name           | Calories (kcal) | Carbohydrates (g) | Protein (g) | Fats (g) | Free Sugar (g) | Fibre (g) | Sodium (mg) | Cal |
|-----|---------------------|-----------------|-------------------|-------------|----------|----------------|-----------|-------------|-----|
| 666 | Fish pakora/ pakoda | 577.03          | 4.55              | 5.97        | 59.52    | 0.22           | 0.96      | 112.24      |     |

In [6]: `df.shape`

Out[6]: (1014, 12)

In [7]: `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1014 entries, 0 to 1013
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Dish Name             1014 non-null   object
1   Calories (kcal)        1014 non-null   float64
2   Carbohydrates (g)     1014 non-null   float64
3   Protein (g)           1014 non-null   float64
4   Fats (g)              1014 non-null   float64
5   Free Sugar (g)        1014 non-null   float64
6   Fibre (g)             1014 non-null   float64
7   Sodium (mg)           1014 non-null   float64
8   Calcium (mg)          1014 non-null   float64
9   Iron (mg)             1014 non-null   float64
10  Vitamin C (mg)        932 non-null    float64
11  Folate (µg)           932 non-null    float64
dtypes: float64(11), object(1)
memory usage: 95.2+ KB
```

In [8]: `df.describe()`

Out[8]:

|              | Calories<br>(kcal) | Carbohydrates<br>(g) | Protein (g) | Fats (g)    | Free Sugar<br>(g) | F    |
|--------------|--------------------|----------------------|-------------|-------------|-------------------|------|
| <b>count</b> | 1014.000000        | 1014.000000          | 1014.000000 | 1014.000000 | 1014.000000       | 1014 |
| <b>mean</b>  | 233.735897         | 18.354004            | 4.753008    | 16.267584   | 8.824832          | 1    |
| <b>std</b>   | 187.599467         | 16.745936            | 3.442746    | 20.452322   | 12.378324         | 2    |
| <b>min</b>   | 6.610000           | 0.000000             | 0.000000    | 0.000000    | 0.000000          | 0    |
| <b>25%</b>   | 101.180000         | 5.582500             | 2.100000    | 4.030000    | 1.152500          | 0    |
| <b>50%</b>   | 174.920000         | 11.925000            | 3.990000    | 9.095000    | 2.780000          | 1    |
| <b>75%</b>   | 315.845000         | 26.870000            | 6.602500    | 17.085000   | 11.822500         | 2    |
| <b>max</b>   | 839.330000         | 86.530000            | 21.550000   | 90.450000   | 85.570000         | 35   |

In [ ]:

## Checking Null Values

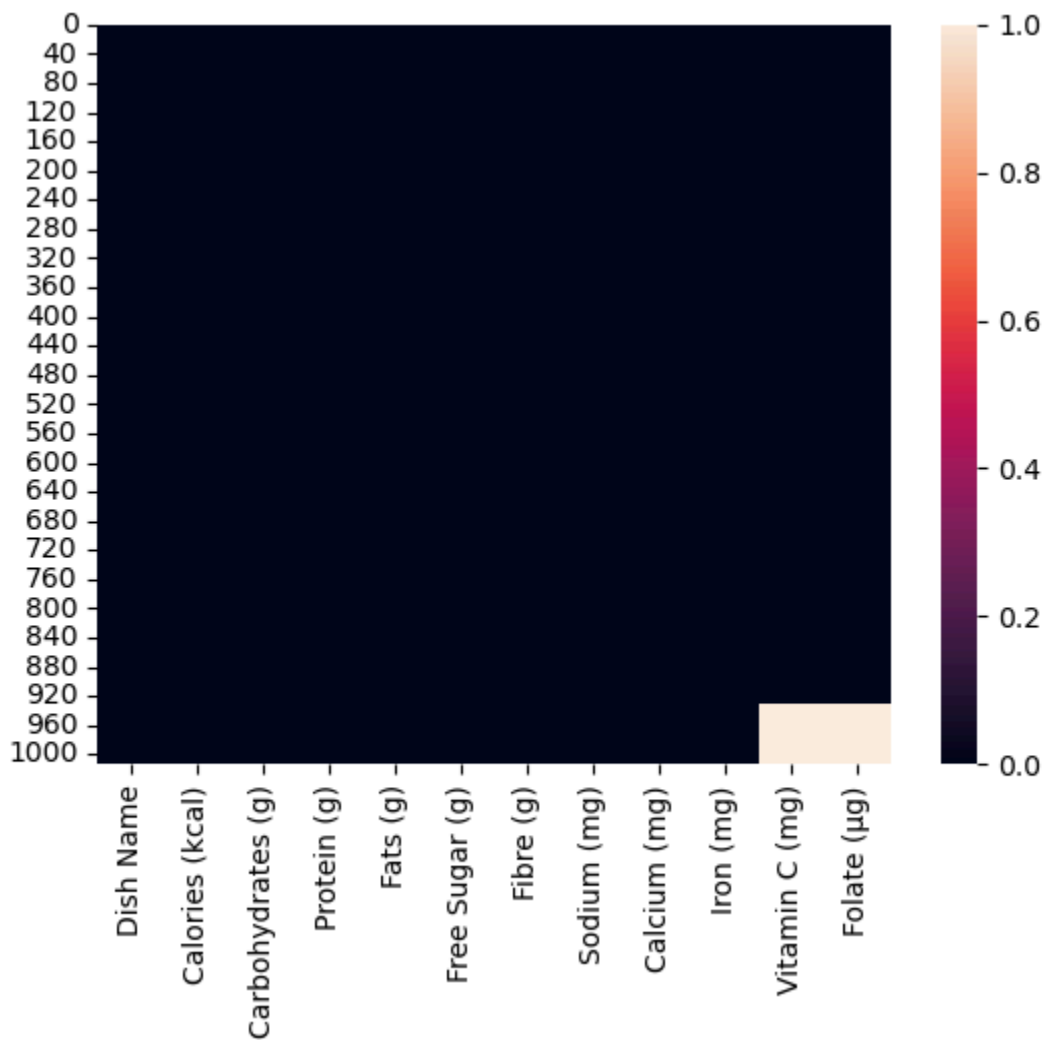
Missing values are identified to ensure data quality before training the model.

In [9]: `df.isnull().sum()`

```
Out[9]: Dish Name      0
        Calories (kcal) 0
        Carbohydrates (g) 0
        Protein (g)      0
        Fats (g)         0
        Free Sugar (g)    0
        Fibre (g)        0
        Sodium (mg)      0
        Calcium (mg)     0
        Iron (mg)        0
        Vitamin C (mg)   82
        Folate (µg)     82
        dtype: int64
```

```
In [10]: sns.heatmap(df.isnull())
```

```
Out[10]: <Axes: >
```



```
In [11]: df.dtypes
```

```
Out[11]: Dish Name      object
         Calories (kcal) float64
         Carbohydrates (g) float64
         Protein (g)    float64
         Fats (g)       float64
         Free Sugar (g) float64
         Fibre (g)      float64
         Sodium (mg)    float64
         Calcium (mg)   float64
         Iron (mg)      float64
         Vitamin C (mg) float64
         Folate (µg)    float64
         dtype: object
```

```
In [12]: df.columns
```

```
Out[12]: Index(['Dish Name', 'Calories (kcal)', 'Carbohydrates (g)', 'Protein (g)',
               'Fats (g)', 'Free Sugar (g)', 'Fibre (g)', 'Sodium (mg)',
               'Calcium (mg)', 'Iron (mg)', 'Vitamin C (mg)', 'Folate (µg)'],
              dtype='object')
```

```
In [ ]:
```

## Data Wrangling (Filling Null/Missing Values)

Some numerical features contain missing values which will be handled using median imputation.

```
In [13]: num_cols=['Vitamin C (mg)', 'Folate (µg)']
         df[num_cols] = df[num_cols].fillna(df[num_cols].median())
```

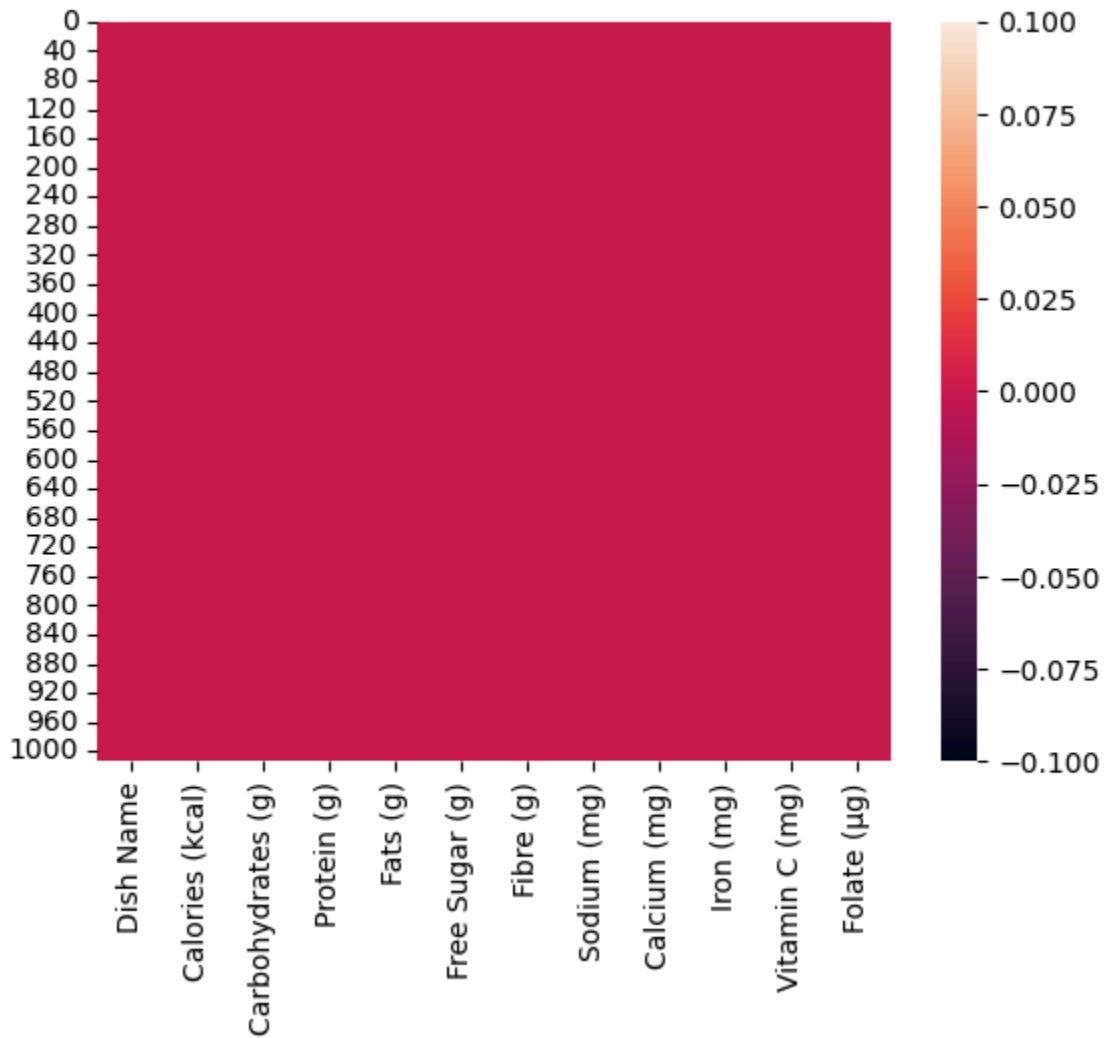
```
In [14]: df.isnull().sum()
```

```
Out[14]: Dish Name      0
         Calories (kcal)  0
         Carbohydrates (g)  0
         Protein (g)      0
         Fats (g)         0
         Free Sugar (g)   0
         Fibre (g)        0
         Sodium (mg)      0
         Calcium (mg)     0
         Iron (mg)        0
         Vitamin C (mg)   0
         Folate (µg)      0
         dtype: int64
```

```
In [15]: sns.heatmap(df.isnull())
```



Out[15]: <Axes: >



```
In [16]: df.isna().mean()*100
```

```
Out[16]: Dish Name          0.0
Calories (kcal)         0.0
Carbohydrates (g)       0.0
Protein (g)             0.0
Fats (g)                0.0
Free Sugar (g)          0.0
Fibre (g)               0.0
Sodium (mg)             0.0
Calcium (mg)            0.0
Iron (mg)               0.0
Vitamin C (mg)          0.0
Folate (µg)             0.0
dtype: float64
```

```
In [17]: df.columns
```

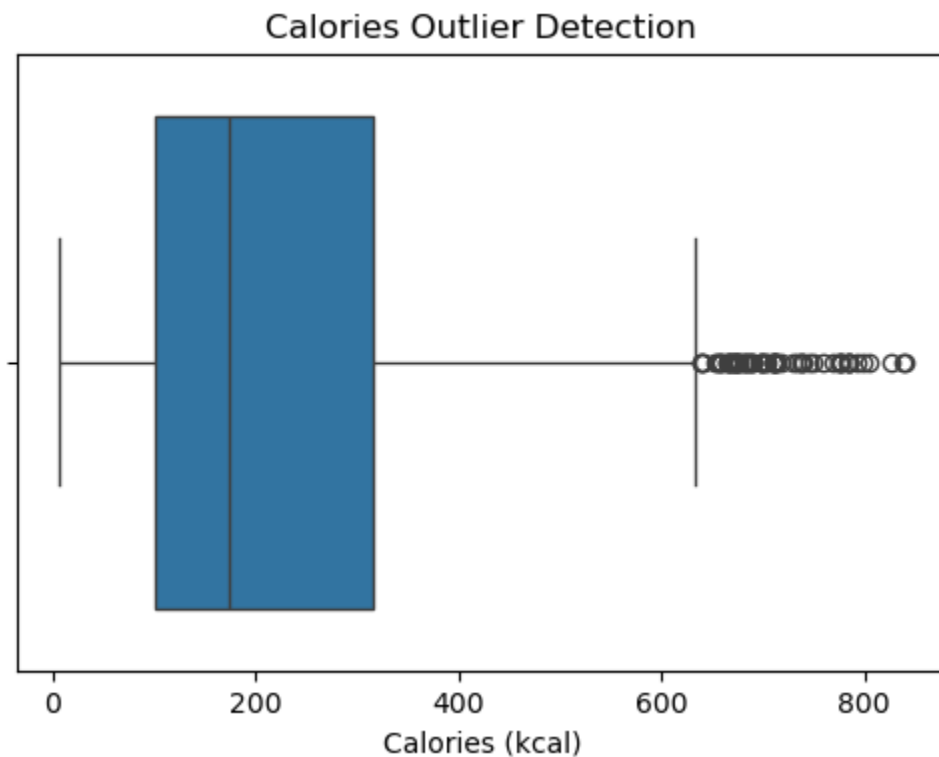
```
Out[17]: Index(['Dish Name', 'Calories (kcal)', 'Carbohydrates (g)', 'Protein (g)',  
              'Fats (g)', 'Free Sugar (g)', 'Fibre (g)', 'Sodium (mg)',  
              'Calcium (mg)', 'Iron (mg)', 'Vitamin C (mg)', 'Folate (µg)'],  
              dtype='object')
```

```
In [ ]:
```

## Data Visualization

```
In [ ]:
```

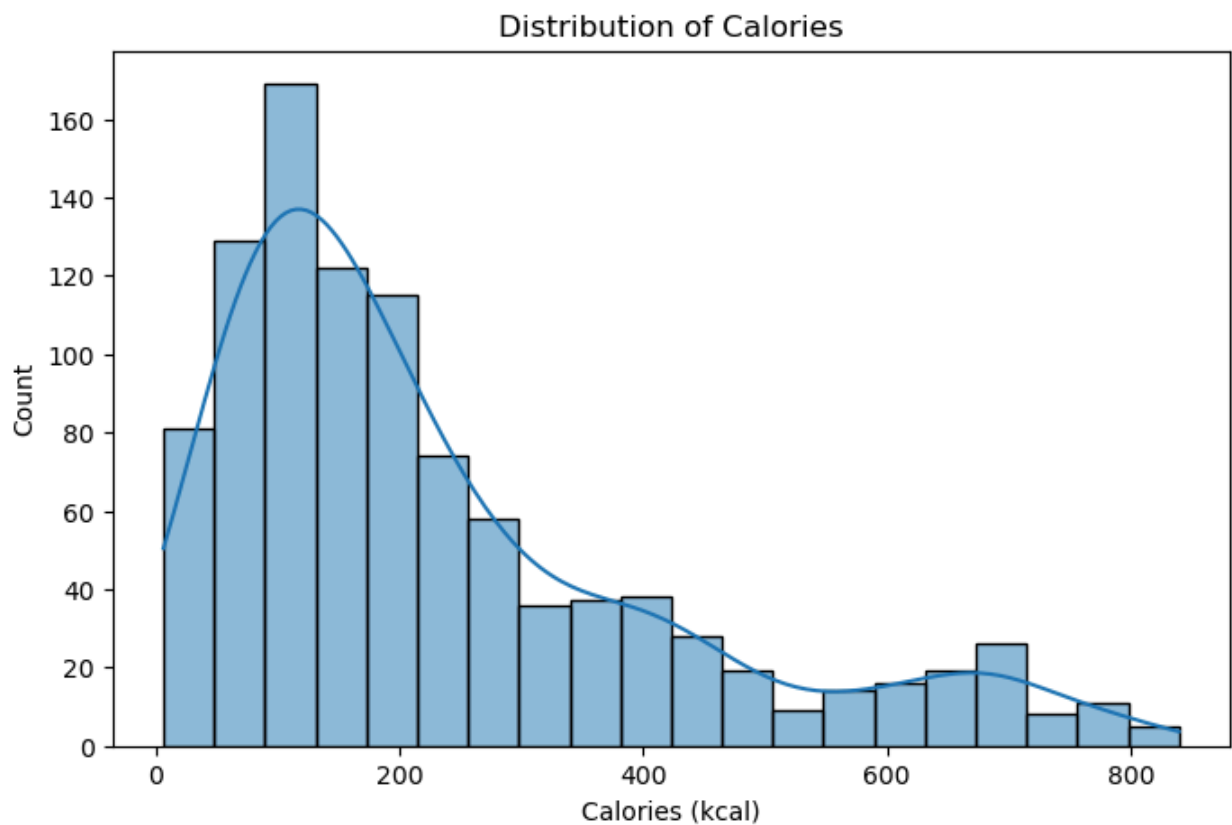
```
In [18]: plt.figure(figsize=(6,4))  
sns.boxplot(x=df['Calories (kcal)'])  
plt.title("Calories Outlier Detection")  
plt.show()
```



Most dishes have calories between 100–300 kcal, while some dishes exceed 600 kcal, indicating high-calorie items.

```
In [ ]:
```

```
In [19]: plt.figure(figsize=(8,5))  
sns.histplot(df['Calories (kcal)'], kde=True)  
plt.title("Distribution of Calories")  
plt.show()
```

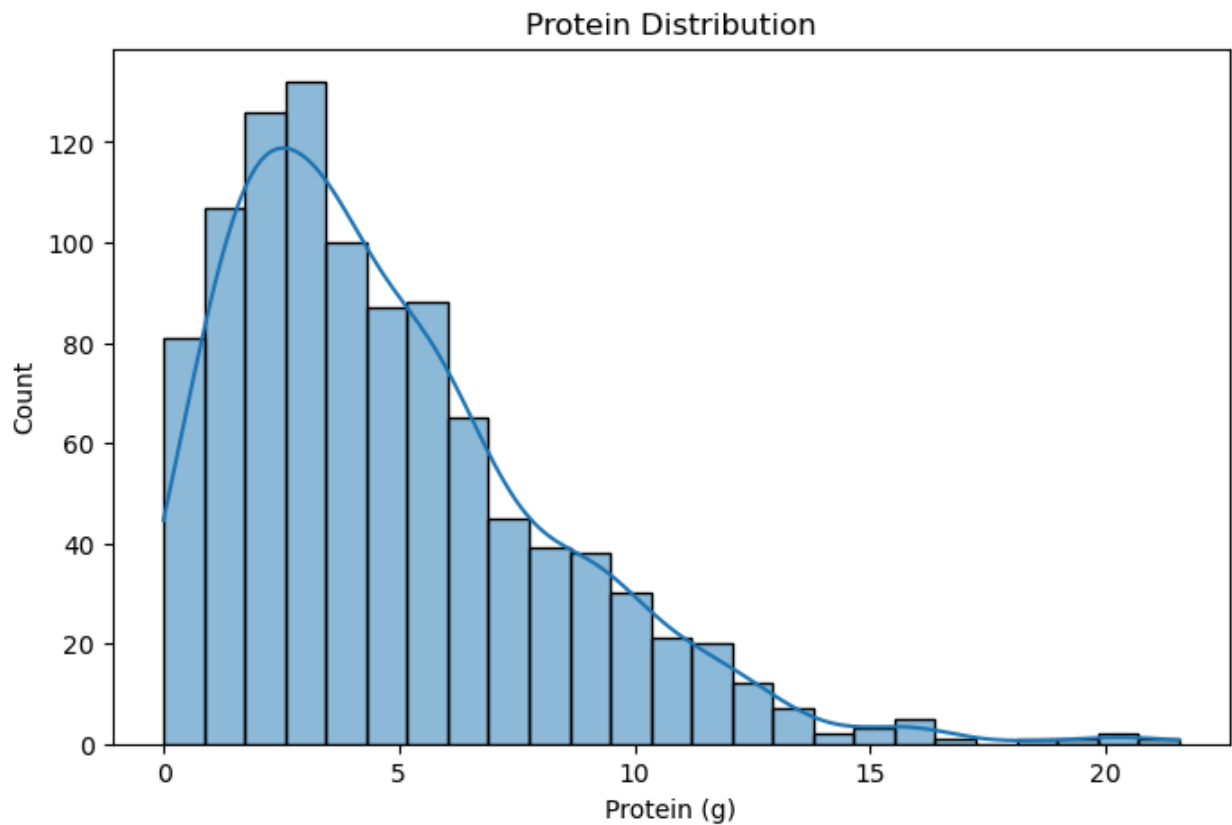


The calorie distribution is right-skewed, meaning most dishes contain lower calories. A smaller number of dishes have very high calorie values, creating a long tail.

```
In [20]: fig = px.scatter(  
    df,  
    x='Fats (g)',  
    y='Calories (kcal)',  
    hover_name='Dish Name',  
    title=("Fats vs Calories"))  
fig.show()
```

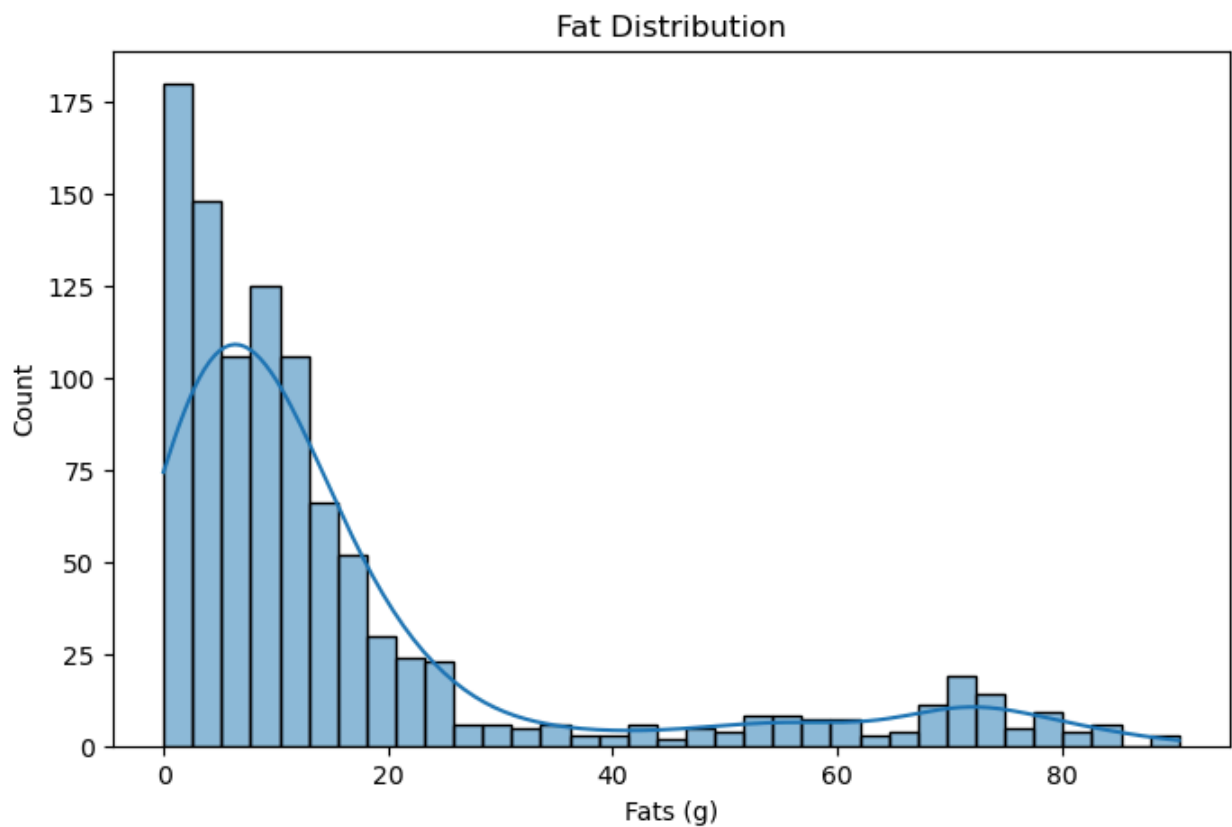
There is a strong positive relationship between fats and calories. As fat content increases, the calorie content also increases significantly.

```
In [21]: plt.figure(figsize=(8,5))
sns.histplot(df['Protein (g)'], kde=True)
plt.title("Protein Distribution")
plt.show()
```



Most dishes contain low to moderate protein levels. Very high protein values are relatively rare in the dataset.

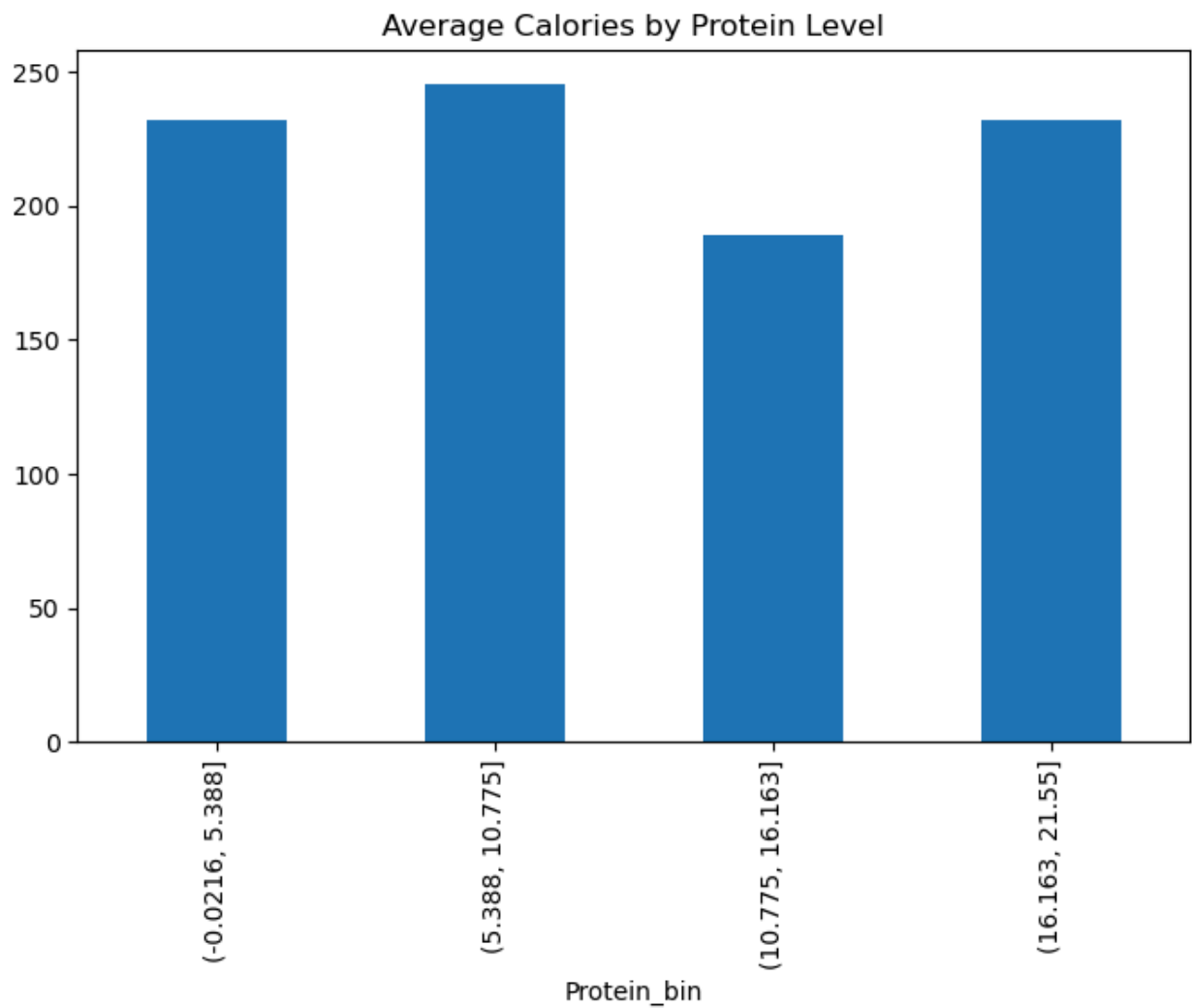
```
In [22]: plt.figure(figsize=(8,5))
sns.histplot(df['Fats (g)'], kde=True)
plt.title("Fat Distribution")
plt.show()
```



Fat values are mostly concentrated at lower levels, but a few dishes contain very high fat values, indicating heavy dishes.

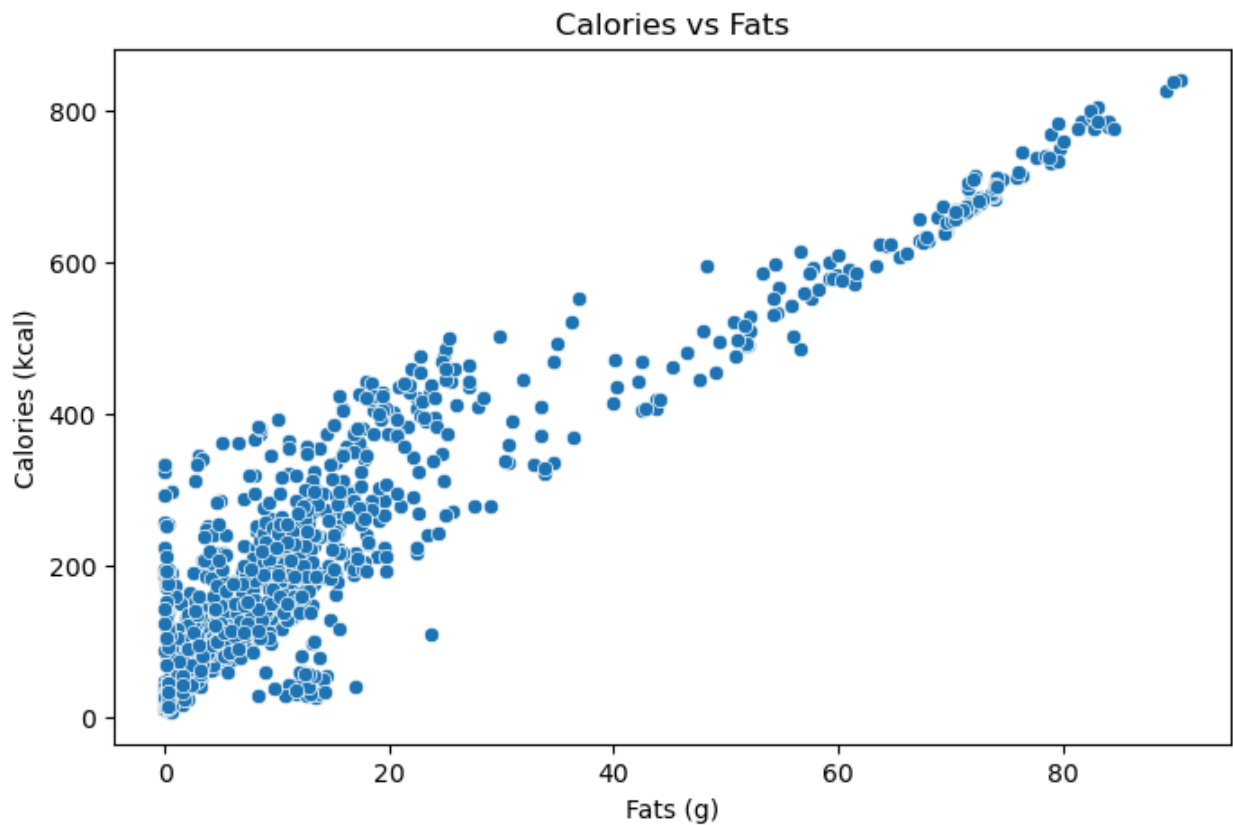
```
In [23]: df['Protein_bin'] = pd.cut(df['Protein (g)'], bins=4)

df.groupby('Protein_bin')['Calories (kcal)'].mean().plot(
    kind='bar', figsize=(8,5)
)
plt.title("Average Calories by Protein Level")
plt.show()
```



The average calories vary across different protein levels. However, protein alone does not strongly determine calorie levels.

```
In [24]: plt.figure(figsize=(8,5))  
sns.scatterplot(x='Fats (g)', y='Calories (kcal)', data=df)  
plt.title("Calories vs Fats")  
plt.show()
```

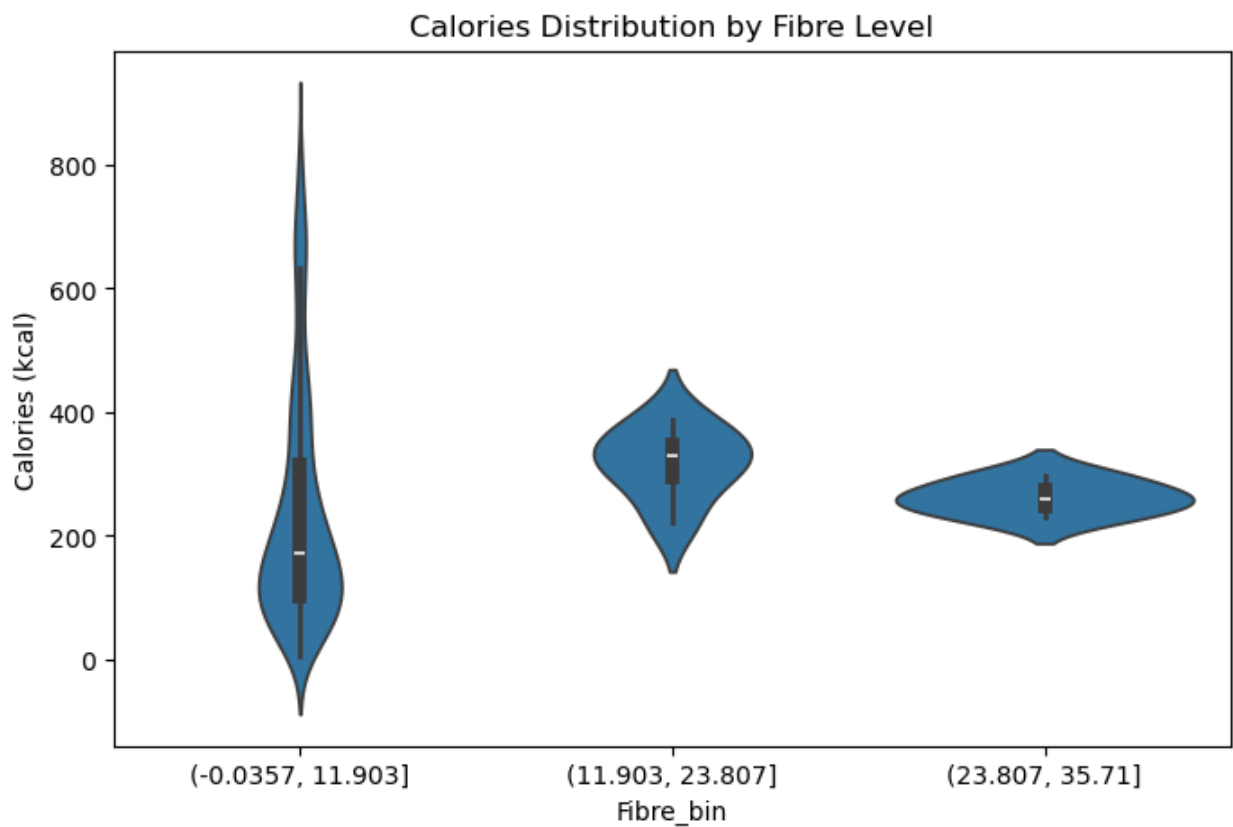


The scatter plot clearly shows that calories increase as fat content increases. Fat appears to be one of the most important contributors to calorie content.

```
In [25]: df['Fibre_bin'] = pd.cut(df['Fibre (g)'], bins=3)

plt.figure(figsize=(8,5))
sns.violinplot(x='Fibre_bin', y='Calories (kcal)', data=df)
plt.title("Calories Distribution by Fibre Level")
plt.show()
```

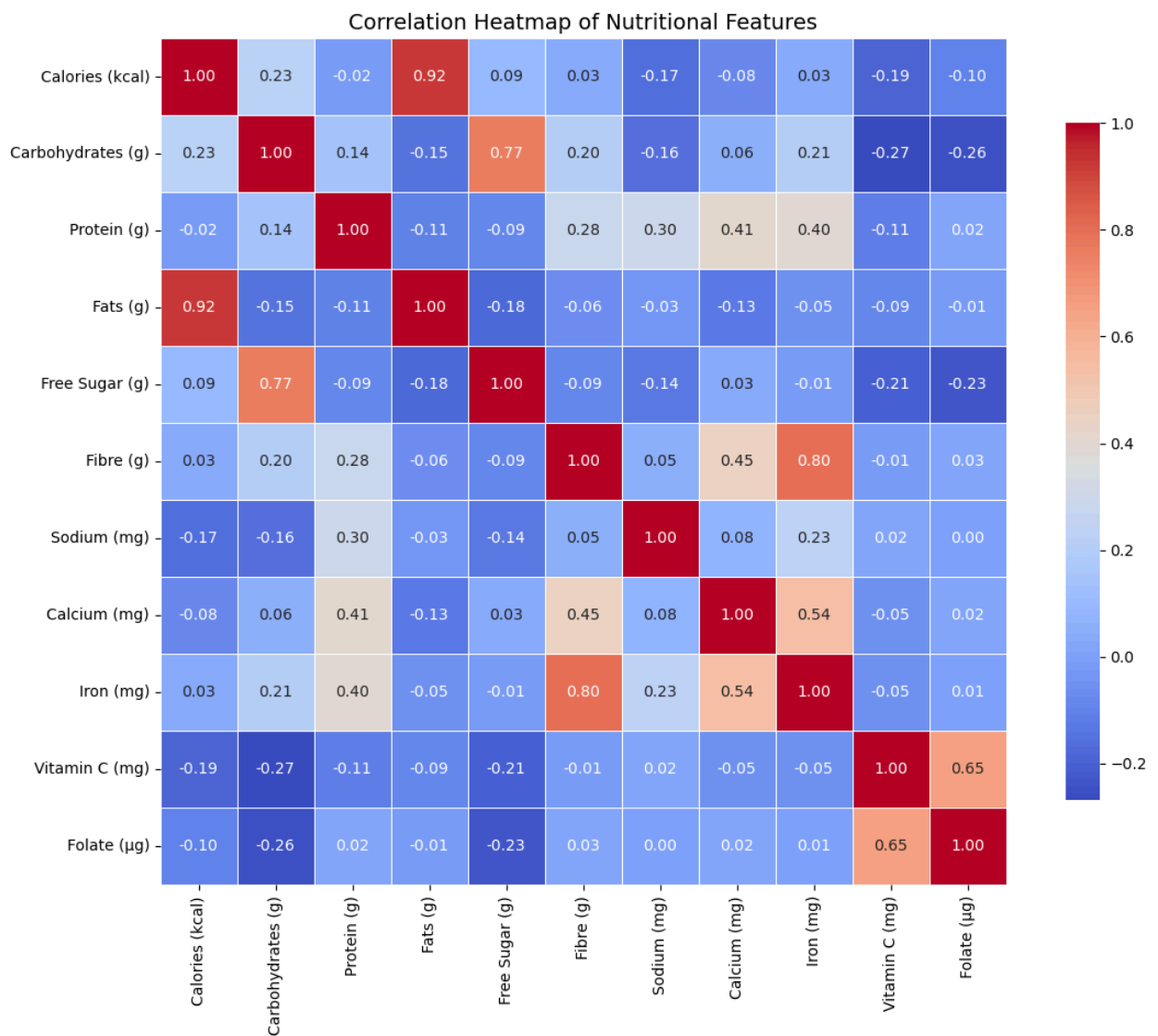




Medium fibre dishes generally have higher calorie levels, while low fibre dishes show a wider variation in calories.

```
In [26]: num_df = df.select_dtypes(include=['int64', 'float64'])
```

```
In [27]: plt.figure(figsize=(14,10))
sns.heatmap(num_df.corr(),annot=True,fmt=".2f",cmap="coolwarm",
            linewidths=0.5,square=True,cbar_kws={"shrink": 0.8})
plt.title("Correlation Heatmap of Nutritional Features", fontsize=14)
plt.show()
```

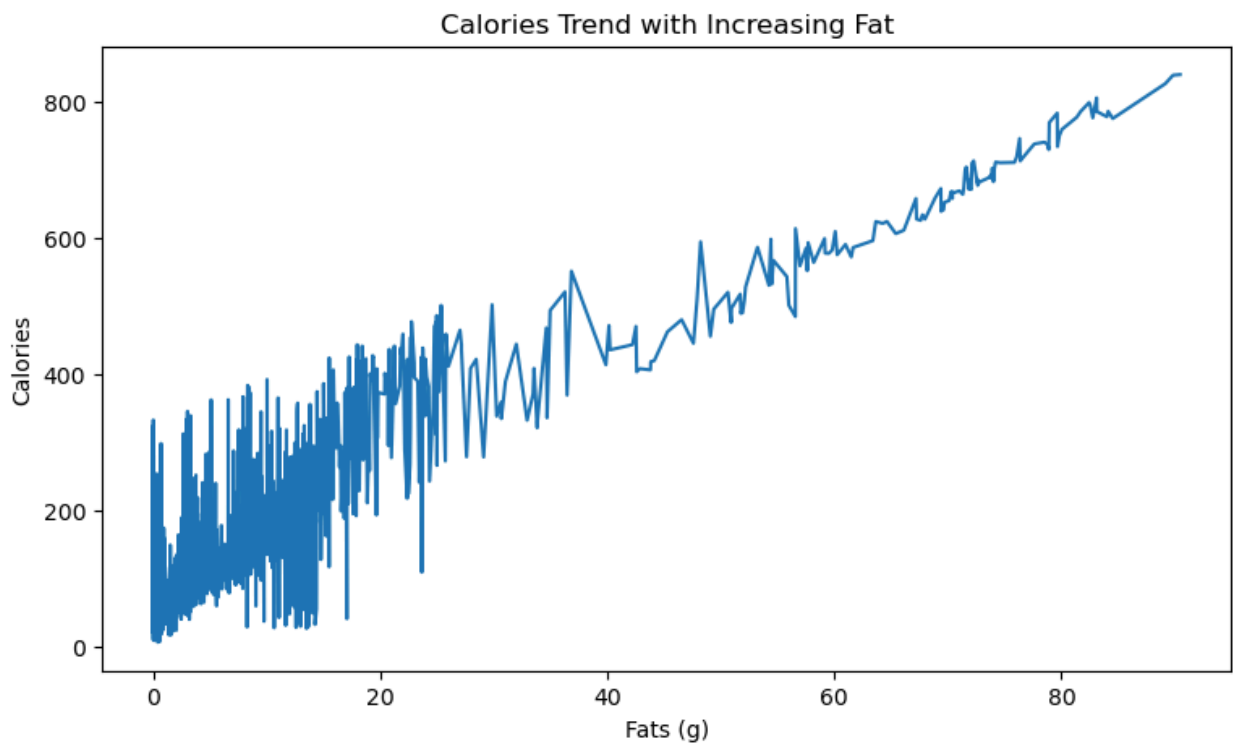


The heatmap shows relationships between nutritional variables. Fat has the strongest positive correlation with calories (0.92). Other nutrients show weak or moderate relationships.

In [ ]:

```
In [28]: df_sorted = df.sort_values('Fats (g)')

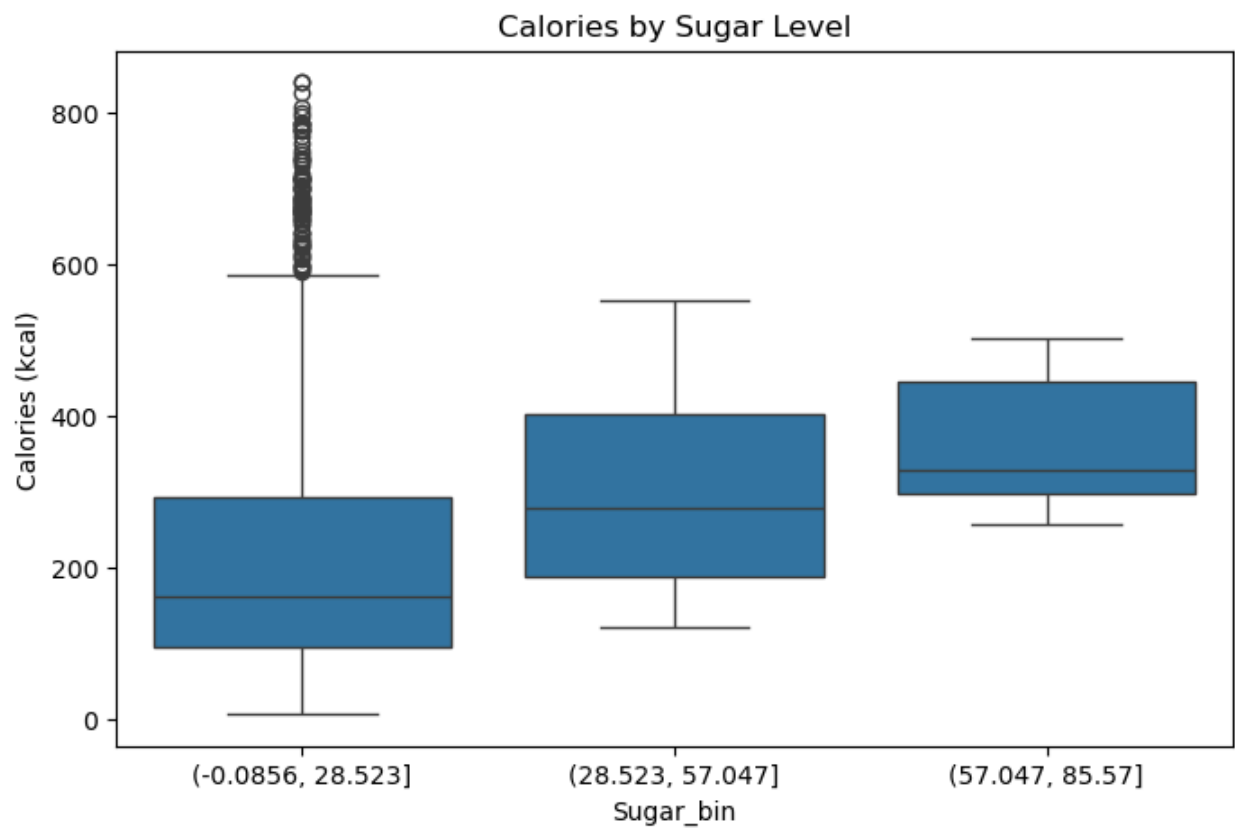
plt.figure(figsize=(9,5))
plt.plot(df_sorted['Fats (g)'], df_sorted['Calories (kcal)'])
plt.title("Calories Trend with Increasing Fat")
plt.xlabel("Fats (g)")
plt.ylabel("Calories")
plt.show()
```



The trend line clearly shows that calories increase steadily with fat content. This confirms the strong relationship between fat and calorie levels.

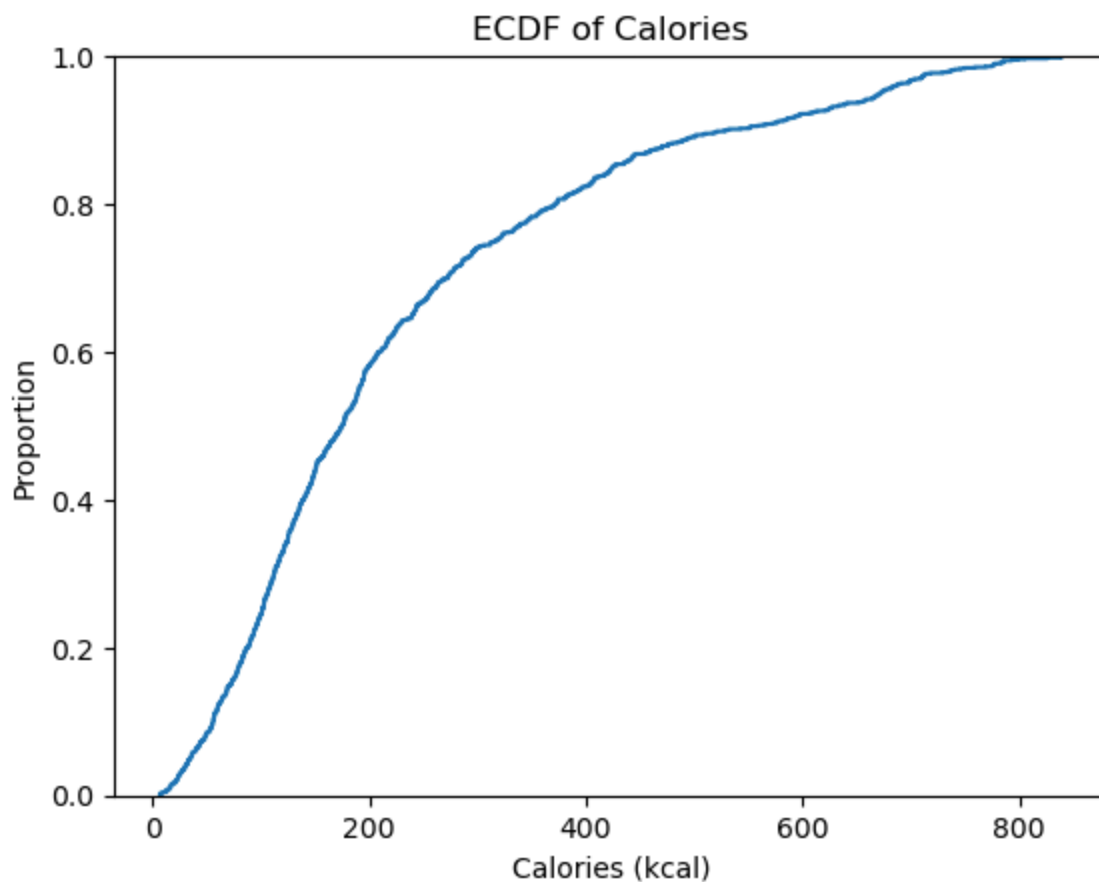
```
In [29]: df['Sugar_bin'] = pd.cut(df['Free Sugar (g)'], bins=3)

plt.figure(figsize=(8,5))
sns.boxplot(x='Sugar_bin', y='Calories (kcal)', data=df)
plt.title("Calories by Sugar Level")
plt.show()
```



Higher sugar levels generally correspond to higher calorie values. However, the relationship is not as strong as fat vs calories.

```
In [30]: sns.ecdfplot(df['Calories (kcal)'])  
plt.title("ECDF of Calories")  
plt.show()
```



The ECDF plot shows the cumulative distribution of calories. Approximately 50% of dishes contain less than around 200–250 kcal.

## Feature Selection (x, y)

```
In [31]: x = df.drop(columns=['Calories (kcal)', 'Dish Name', 'Protein_bin', 'Fibre_bin', '
y = df['Calories (kcal)']
```

## Train-Test Split

The dataset is divided into 80% training data and 20% testing data.

```
In [32]: x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
```

```
In [ ]:
```

# Model Training

```
In [33]: dt = DecisionTreeRegressor(random_state=42)
dt.fit(x_train, y_train)
```

```
Out[33]: DecisionTreeRegressor
DecisionTreeRegressor(random_state=42)
```

# Model Prediction

```
In [34]: y_pred = dt.predict(x_test)
```

```
In [ ]:
```

# Model Evaluation (R2 Score, Mean Absolute Error, Root Mean Squared Error)

```
In [35]: r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
print("R2 :", r2)
print("MAE :", mae)
print("RMSE:", rmse)
```

```
R2 : 0.9870360031397543
MAE : 13.660689655172416
RMSE: 21.5242253254808
```

```
In [ ]:
```

# Checking R2\_Score of Training and Testing

```
In [36]: print("Train R2:", dt.score(x_train, y_train))
print("Test R2:", dt.score(x_test, y_test))
```

```
Train R2: 1.0
Test R2: 0.9870360031397543
```

```
In [ ]:
```

# Pruning

```
In [37]: dt_pruned = DecisionTreeRegressor(max_depth=4,min_samples_leaf=10,random_state=42)
dt_pruned.fit(x_train, y_train)

print("Train R²:", dt_pruned.score(x_train, y_train))
print("Test R²:", dt_pruned.score(x_test, y_test))
```

```
Train R²: 0.9553612941719143
Test R²: 0.9525419415242475
```

In [ ]:

## GridSearchCV

GridSearchCV tests multiple combinations of hyperparameters to find the best K value and distance metric.

```
In [38]: param_grid = {
    'max_depth': [3, 4, 5, 6, 8],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 5, 10]
}
```

```
In [39]: dt_grid = GridSearchCV(estimator=DecisionTreeRegressor(random_state=42), param_
    cv=5, scoring='r2', n_jobs=-1)
dt_grid.fit(x_train, y_train)
```

```
Out[39]:
```

```
GridSearchCV
└─ best_estimator_: DecisionTreeRegressor
    └─ DecisionTreeRegressor
```

```
In [40]: dt_grid.best_params_
```

```
Out[40]: {'max_depth': 8, 'min_samples_leaf': 5, 'min_samples_split': 2}
```

```
In [45]: best_dt_grid = dt_grid.best_estimator_

y_pred_grid = best_dt_grid.predict(x_test)

print("Grid DT MAE :", mean_absolute_error(y_test, y_pred_grid))
print("Grid DT RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_grid)))
```

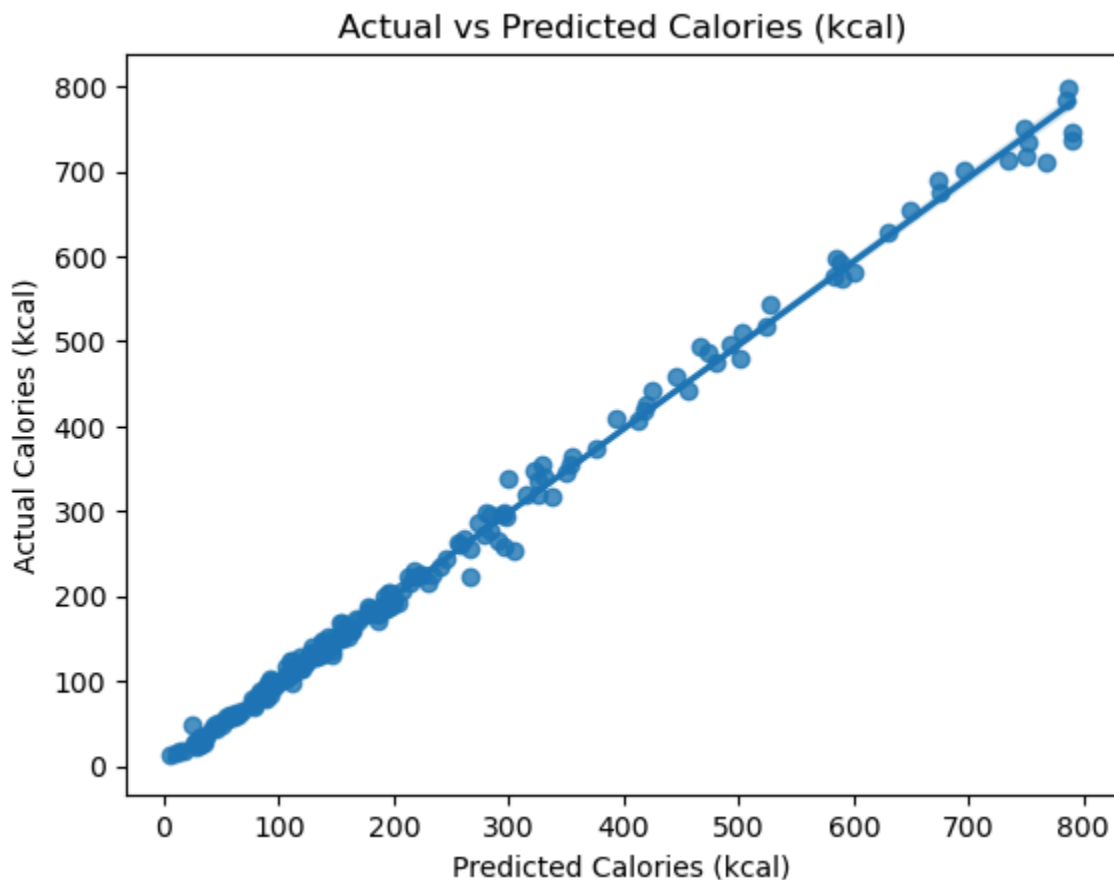
```
print("Grid DT R² :", r2_score(y_test, y_pred_grid))

print("Train R²:", best_dt_grid.score(x_train, y_train))
print("Test R²:", best_dt_grid.score(x_test, y_test))
```

Grid DT MAE : 15.378964299392287  
Grid DT RMSE: 21.047569885966325  
Grid DT R² : 0.9876038227345139  
Train R²: 0.992932922142699  
Test R²: 0.9876038227345139

In [ ]:

```
In [57]: sns.regplot(x=y_pred, y=y_test)
plt.xlabel("Predicted Calories (kcal)")
plt.ylabel('Actual Calories (kcal)')
plt.title("Actual vs Predicted Calories (kcal)")
plt.show()
```



The predicted calorie values closely follow the actual values, indicating that the Decision Tree model provides reasonably accurate predictions.

In [ ]:

In [ ]:



# Improving Model Performance with Advanced Algorithms

In [ ]:

```
In [48]: # Models
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.svm import SVR
from sklearn.neighbors import KNeighborsRegressor
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor

from xgboost import XGBRegressor
from lightgbm import LGBMRegressor

# evaluation
def evaluate(y_true, y_pred):
    r2 = r2_score(y_true, y_pred)
    mae = mean_absolute_error(y_true, y_pred)
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    return r2, mae, rmse

results = []

# Linear Regression
lr = LinearRegression()
lr.fit(x_train, y_train)
y_pred = lr.predict(x_test)
results.append(["Linear Regression", *evaluate(y_test, y_pred)])

# Ridge
ridge = GridSearchCV(
    Ridge(),
    {"alpha": [0.1, 1]},
    cv=2,
    n_jobs=-1
)
ridge.fit(x_train, y_train)
y_pred = ridge.predict(x_test)
results.append(["Ridge", *evaluate(y_test, y_pred)])

# KNN
knn = GridSearchCV(
    KNeighborsRegressor(),
    {"n_neighbors": [3, 5]},
    cv=2,
    n_jobs=-1
)
knn.fit(x_train, y_train)
y_pred = knn.predict(x_test)
results.append(["KNN Regressor", *evaluate(y_test, y_pred)])
```

```

# SVR
svr = GridSearchCV(
    SVR(),
    {"C": [1], "kernel": ["rbf"]},
    cv=2,
    n_jobs=-1
)
svr.fit(x_train, y_train)
y_pred = svr.predict(x_test)
results.append(["SVR", *evaluate(y_test, y_pred)])

# Random Forest
rf = GridSearchCV(
    RandomForestRegressor(),
    {"n_estimators": [100], "max_depth": [None, 10]},
    cv=2,
    n_jobs=-1
)
rf.fit(x_train, y_train)
y_pred = rf.predict(x_test)
results.append(["Random Forest", *evaluate(y_test, y_pred)])

# Gradient Boosting (IMPORTANT)
gb = GridSearchCV(
    GradientBoostingRegressor(),
    {"n_estimators": [100], "learning_rate": [0.1]},
    cv=2,
    n_jobs=-1
)
gb.fit(x_train, y_train)
y_pred = gb.predict(x_test)
results.append(["Gradient Boosting", *evaluate(y_test, y_pred)])

# XGBoost
xgb = XGBRegressor(n_estimators=100, learning_rate=0.1)
xgb.fit(x_train, y_train)
y_pred = xgb.predict(x_test)
results.append(["XGBoost", *evaluate(y_test, y_pred)])

# LightGBM
lgb = LGBMRegressor(n_estimators=100, learning_rate=0.1, verbose=-1)
lgb.fit(x_train, y_train)
y_pred = lgb.predict(x_test)
results.append(["LightGBM", *evaluate(y_test, y_pred)])

# TABLE
columns = ["Model", "R2 Score", "MAE", "RMSE"]

df_reg = pd.DataFrame(results, columns=columns)
df_reg = df_reg.sort_values(by="R2 Score", ascending=False)

```

```
display(df_reg)
```

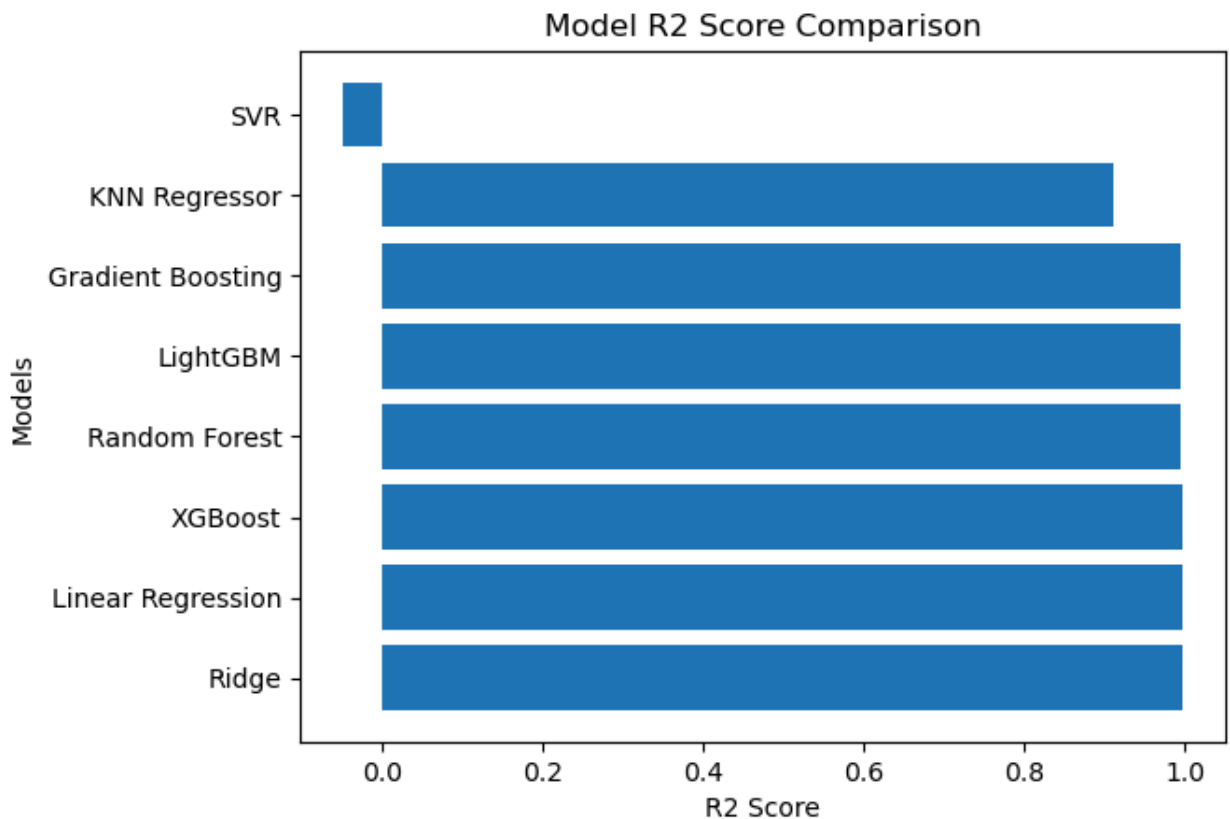
|   | Model             | R2 Score  | MAE        | RMSE       |
|---|-------------------|-----------|------------|------------|
| 1 | Ridge             | 0.998636  | 2.834785   | 6.981718   |
| 0 | Linear Regression | 0.998636  | 2.834090   | 6.982081   |
| 6 | XGBoost           | 0.996945  | 6.866589   | 10.449492  |
| 4 | Random Forest     | 0.996086  | 7.940140   | 11.827565  |
| 7 | LightGBM          | 0.995833  | 7.684368   | 12.202374  |
| 5 | Gradient Boosting | 0.995790  | 8.463648   | 12.265248  |
| 2 | KNN Regressor     | 0.911732  | 38.355074  | 56.164085  |
| 3 | SVR               | -0.049759 | 132.552167 | 193.688034 |

```
In [ ]:
```

```
In [52]: plt.figure()
plt.barh(df_reg["Model"], df_reg["R2 Score"])

plt.xlabel("R2 Score")
plt.ylabel("Models")
plt.title("Model R2 Score Comparison")

plt.show()
```



All models except SVR achieve very high  $R^2$  scores, with ensemble methods (Random Forest, XGBoost, LightGBM, Gradient Boosting) performing the best, while SVR performs the worst.

In [ ]: